```
# Check GPU is working
!nvidia-smi

# Find the exact torch + CUDA version (needed for PyG)
import torch
print("Torch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

```
Fri Oct  9 19:12:26 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   46C    P8             12W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
Torch version: 2.11.0+cu130
CUDA available: True
```

```
# Modern PyG doesn't need torch-scatter/sparse to be compiled from source
# Just install torch-geometric directly — works on Colab with no build step

!pip install -q torch-geometric

!pip install -q transformers datasets rapidfuzz networkx rdflib \
    pandas numpy scikit-learn scipy rdkit pubchempy \
    pyyaml omegaconf matplotlib seaborn plotly \
    tqdm loguru rich python-dotenv pytest
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 99.5 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 615.4/615.4 kB 51.3 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 31.8/31.8 MB 73.0 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 61.6/61.6 kB 6.8 MB/s eta 0:00:00
```

```
import torch_geometric
import torch
print("PyG:", torch_geometric.__version__)
print("CUDA:", torch.cuda.is_available())
print("✅ Ready!")
```

```
PyG: 2.8.0.post1
CUDA: True
✅ Ready!
```

```
import os

!git clone https://github.com/jiya2000/Reliability-Conditioned-Herb-Drug-Interaction-Prediction.git
```

```
# Move into the project folder
os.chdir("Reliability-Conditioned-Herb-Drug-Interaction-Prediction")

# Create all required folders
!mkdir -p logs results/figures checkpoints/ablation data/processed data/raw

# Confirm everything is there
print("✅ Working directory:", os.getcwd())
!ls
```

```
Cloning into 'Reliability-Conditioned-Herb-Drug-Interaction-Prediction'...
remote: Enumerating objects: 211, done.
remote: Counting objects: 100% (211/211), done.
remote: Compressing objects: 100% (151/151), done.
remote: Total 211 (delta 82), reused 181 (delta 52), pack-reused 0 (from 0)
Receiving objects: 100% (211/211), 7.41 MiB | 5.86 MiB/s, done.
Resolving deltas: 100% (82/82), done.
✅ Working directory: /content/Reliability-Conditioned-Herb-Drug-Interaction-Prediction/Reliability-Cond
 app                                        README.md
 checkpoints                                requirements_core.txt
 configs                                    requirements.txt
 create_pdf.py                              results
 data                                       Review_1_Report_Final.md
 docs                                       Review_1_Report_Final.pdf
 generate_architecture_diagram.py           Review_1_Report.md
'HDI_Final_Implementation_Plan (1).md'      Review_1_Report.pdf
 implementation_plan.md                     run_experiments.py
 LICENSE                                   'Screenshot 2026-07-23 230057.png'
 LITERATURE_REVIEW_AND_PROJECT_GUIDE.md     src
 logs                                       STATUS_AND_PROGRESS.md
 main.py                                    tests
 pyproject.toml                             uv.lock
```

```
!python run_experiments.py \
    --mode synthetic \
    --epochs 30 \
    --seeds 42 123 456 \
    --hidden-dim 128 \
    --num-heads 4 \
    --batch-size 64
```

```
20:39:00 | INFO     | Early stopping at epoch 15 (patience=10)
20:39:00 | INFO     | Training complete. Best val loss: 0.3658
20:39:00 | INFO     |   Training complete in 418.6s
20:39:01 | INFO     | Evaluation Results:
20:39:01 | INFO     |   auc_roc: 0.7502
20:39:01 | INFO     |   auc_pr: 0.3188
20:39:01 | INFO     |   precision: 0.4286
20:39:01 | INFO     |   recall: 0.0750
20:39:01 | INFO     |   f1: 0.1277
20:39:01 | INFO     |   mrr: 0.5763
20:39:01 | INFO     |   hits@10: 1.0000
20:39:01 | INFO     |   hits@5: 0.9710
20:39:01 | INFO     |   hits@1: 0.3188
20:39:01 | INFO     |   num_samples: 480
20:39:01 | INFO     |   num_positives: 80
20:39:01 | INFO     |   num_negatives: 400
20:39:01 | INFO     |   threshold: 0.5000
20:39:03 | INFO     | Stratified Evaluation Results:
20:39:03 | INFO     |   Overall: AUC=0.7502, F1=0.1277
20:39:03 | INFO     |   R_0.25_0.50: AUC=0.7467, F1=0.1205, n=420
20:39:03 | INFO     |   R_0.50_0.75: AUC=0.7640, F1=0.1818, n=60
20:39:04 | INFO     |
============================================================
20:39:04 | INFO     | PHASE 2: Computing aggregate metrics
20:39:04 | INFO     | ============================================================
```

```
20:39:04 | INFO     |   gnn_only: AUC-ROC=0.7568±0.0072, F1=0.1388±0.0383
20:39:04 | INFO     |   unconditioned: AUC-ROC=0.7554±0.0014, F1=0.0974±0.0127
20:39:04 | INFO     |   full_model: AUC-ROC=0.7538±0.0025, F1=0.1003±0.0265
20:39:04 | INFO     |
============================================================
20:39:04 | INFO     | PHASE 3: Running statistical significance tests
20:39:04 | INFO     | ============================================================
20:39:04 | INFO     |
============================================================
20:39:04 | INFO     | PHASE 4: Generating paper-ready tables
20:39:04 | INFO     | ============================================================
20:39:04 | INFO     | Saved JSON results: results/ablation_results.json
20:39:04 | INFO     | Saved paper tables: results/paper_tables.md
20:39:04 | INFO     | Saved full results: results/full_results.json
20:39:04 | INFO     |
============================================================
20:39:04 | INFO     | PHASE 5: Generating paper figures
20:39:04 | INFO     | ============================================================
20:39:05 | INFO     |   Generated: ablation_comparison.png/svg
20:39:06 | INFO     |   Generated: loss_curves.png/svg
20:39:06 | INFO     |   Generated: reliability_calibration.png/svg
20:39:06 | INFO     |   Generated: prediction_calibration.png/svg
20:39:06 | INFO     | All figures generated successfully
20:39:06 | INFO     |
============================================================
20:39:06 | INFO     | EXPERIMENT PIPELINE COMPLETE
20:39:06 | INFO     | ============================================================
20:39:06 | INFO     | Total time: 2675.7s (44.6 min)
20:39:06 | INFO     | Results saved to: results/
20:39:06 | INFO     |   - ablation_results.json  (machine-readable)
20:39:06 | INFO     |   - paper_tables.md        (paper-ready tables)
20:39:06 | INFO     |   - full_results.json      (with loss histories)
20:39:06 | INFO     |   - figures/               (PNG + SVG)
20:39:06 | INFO     |
```

```
# Print results table
with open("results/paper_tables.md") as f:
    print(f.read())

# Also print raw JSON
import json
with open("results/ablation_results.json") as f:
    data = json.load(f)

print("\n📊 Key metrics:")
for variant, metrics in data.get("results", {}).items():
    if isinstance(metrics, dict):
        auc = metrics.get("auc_roc", metrics.get("auc", "N/A"))
        print(f"  {variant}: AUC = {auc}")
```

```
# Experimental Results — HDI Prediction

**Generated**: 2026-10-09 20:39:04
**Data mode**: synthetic
**Seeds**: [42, 123, 456]
**Epochs**: 30
**Gating mode**: multiplicative

## Table 1: Ablation Study Results

| Variant | AUC-ROC | AUC-PR | Precision | Recall | F1 | MRR | Hits@10 | Params |
|---------|---------|--------|-----------|--------|-----|-----|---------|--------|
| (a) GNN-only | 0.7568±0.0072 | 0.3163±0.0020 | 0.3645±0.1423 | 0.0917±0.0328 | 0.1388±0.0383 | 0.6043±0
| (b) Unconditioned | 0.7554±0.0014 | 0.3137±0.0016 | 0.3379±0.0639 | 0.0583±0.0118 | 0.0974±0.0127 | 0.0
| (c) **Full R-conditioned (Ours)** | 0.7538±0.0025 | 0.3071±0.0114 | 0.3587±0.0906 | 0.0583±0.0156 | 0.1

## Table 2: Critical Comparison — (b) vs (c)
```

```
The (b)-vs-(c) gap is the evidentiary core of the paper and patent.

| Metric | Unconditioned (b) | Full Model (c) | Δ (c-b) | Improvement |
|--------|-------------------|----------------|---------|-------------|
| auc_roc | 0.7554 | 0.7538 | ✗ -0.0017 | -0.2% |
| auc_pr | 0.3137 | 0.3071 | ✗ -0.0067 | -2.1% |
| f1 | 0.0974 | 0.1003 | ✓ +0.0029 | +3.0% |
| mrr | 0.6152 | 0.5772 | ✗ -0.0380 | -6.2% |
| hits@10 | 1.0000 | 1.0000 | ✗ +0.0000 | +0.0% |

## Table 3: Statistical Significance

- **AUC-ROC gap (c-b)**: -0.0017
- **Direction**: unconditioned > full
- **Per-seed differences**: [-0.0009062499999999973, 0.0018437500000000329, -0.005906250000000002]

## Table 4: Model Configuration

| Setting | Value |
|---------|-------|
| Data mode | synthetic |
| Epochs | 30 |
| Learning rate | 0.0001 |
| Batch size | 64 |
| Hidden dim | 128 |
| Attention heads | 4 |
| Gating mode | multiplicative |
| Seeds | [42, 123, 456] |


📊 Key metrics:
```

```
from google.colab import files
import shutil

# Zip the results folder
shutil.make_archive("/content/hdi_results", "zip",
                    "/content/Reliability-Conditioned-Herb-Drug-Interaction-Prediction/results")

# Download it
files.download("/content/hdi_results.zip")
```

```
from google.colab import drive
import shutil

# Mount Google Drive
drive.mount('/content/drive')

# Copy results to Drive
shutil.copytree(
    "results",
    "/content/drive/MyDrive/HDI_Patent_Results",
    dirs_exist_ok=True
)

print("✅ Saved to Google Drive → HDI_Patent_Results/")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive
✅ Saved to Google Drive → HDI_Patent_Results/
```

Start coding or generate with AI.